



Cursores

Universidad Nacional Autónoma de México

Bases de Datos. Grupo 1

- Alvarez Salgado Eduardo Antonio
- Merida Serralde Francisco Jared

20 de Mayo de 2026

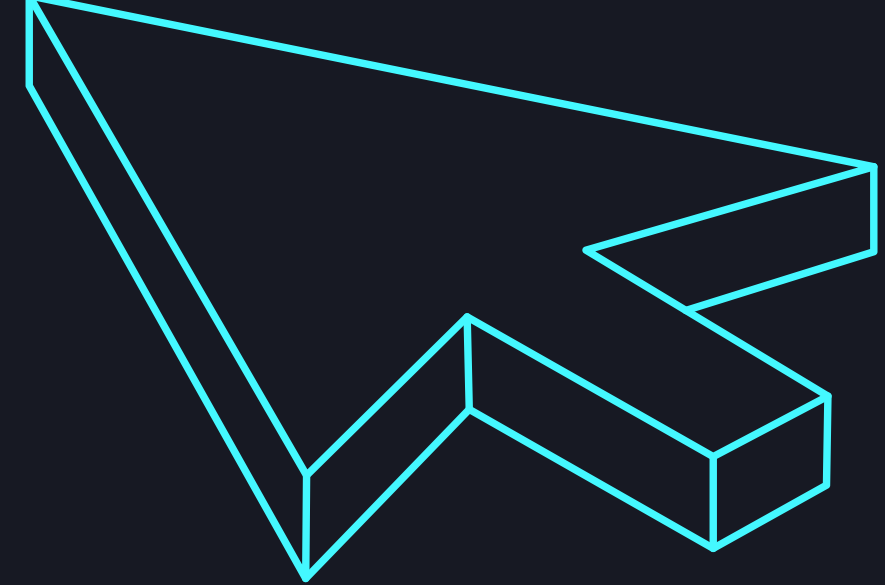


¿CÓMO TRABAJAN LOS MANEJADORES EN BASES DE DATOS RELACIONALES?

CONJUNTOS DE DATOS

Una consulta puede leer, actualizar o eliminar varias filas al mismo tiempo.
Por ejemplo:

```
UPDATE empleados  
SET salario = salario * 1.10  
WHERE departamento = 'Ventas';
```



CURSORES

Un cursor es un mecanismo que permite recorrer el resultado de una consulta fila por fila, en lugar de trabajar con todo el conjunto de datos al mismo tiempo.

Se posiciona sobre una fila específica de un conjunto de resultados y permite avanzar, leer o modificar registros de manera secuencial.

PRINCIPALES USOS:

- Ejecutar una acción distinta según el resultado obtenido en cada fila.
- Recorrer resultados grandes por partes, sin cargar toda la información de una sola vez.
- Procesar filas en un orden específico, por ejemplo, de acuerdo con una fecha o prioridad.
- Se requiere aplicar validaciones individuales sobre cada registro antes de continuar con el siguiente.



TIPOS DE CURSORES



CURSOR EXPLÍCITO

Es el cursor que el programador declara directamente. Se usa cuando se quiere tener control sobre el recorrido de una consulta.

CURSOR IMPLÍCITO

Es el cursor que el manejador controla automáticamente sin que el usuario lo declare de forma directa. Por ejemplo, cuando se usa un ciclo FOR para recorrer los resultados de una consulta, PostgreSQL se encarga internamente del recorrido.

CURSOR VINCULADO O BOUND CURSOR

Es un cursor cuya consulta queda definida desde su declaración.

TIPOS DE CURSORES



CURSOR NO VINCULADO O UNBOUND CURSOR

Es un cursor declarado como variable **refcursor**, pero sin una consulta fija al inicio. La consulta se asigna hasta el momento de abrirlo.

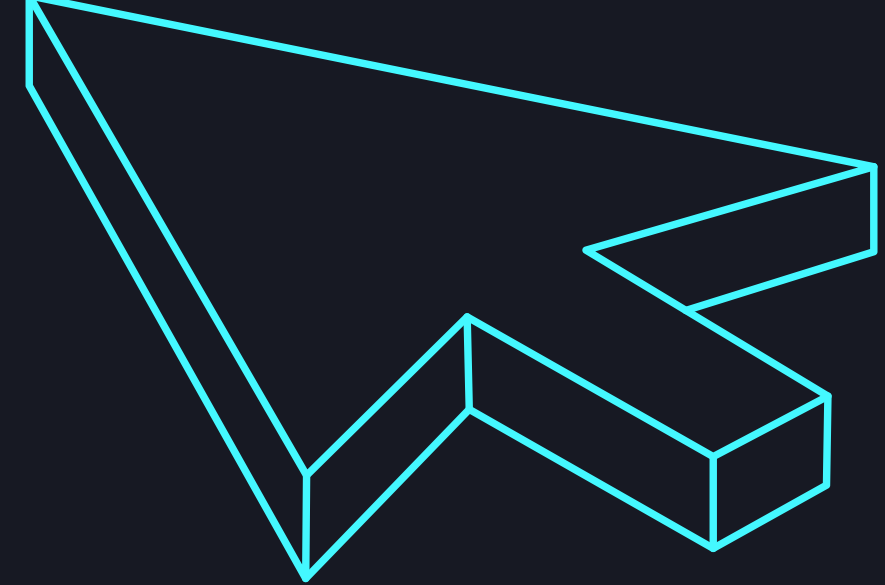
CURSOR CON PARÁMETROS

Es un cursor que recibe valores al momento de abrirse. Esto permite reutilizar la misma estructura del cursor con diferentes condiciones.

CURSOR SCROLL

SCROLL permite moverse hacia adelante y hacia atrás en el resultado.

TIPOS DE CURSORES



CURSOR NO SCROLL

NO SCROLL solo permite avanzar hacia adelante en el resultado.

CURSOR WITH HOLD

WITH HOLD puede mantenerse abierto después de finalizar la transacción con **COMMIT**.

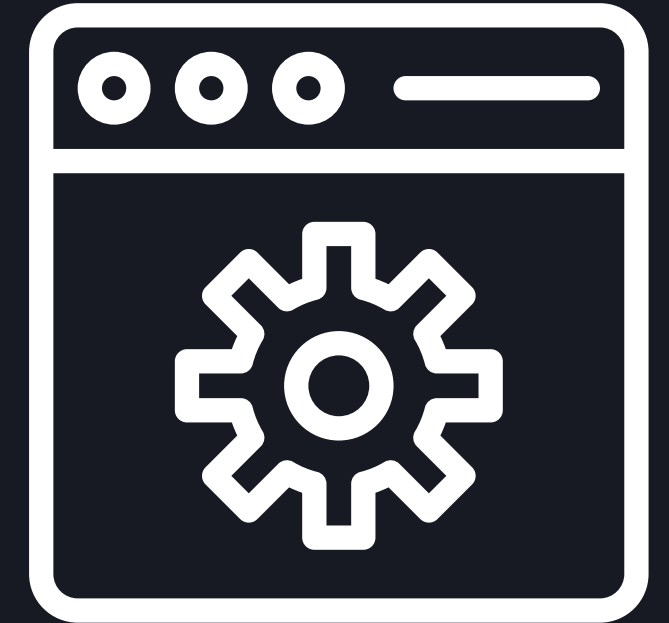
CURSOR WITHOUT HOLD

Es el comportamiento normal. El cursor se cierra cuando termina la transacción.

DECLARACIÓN DE UN CURSOR

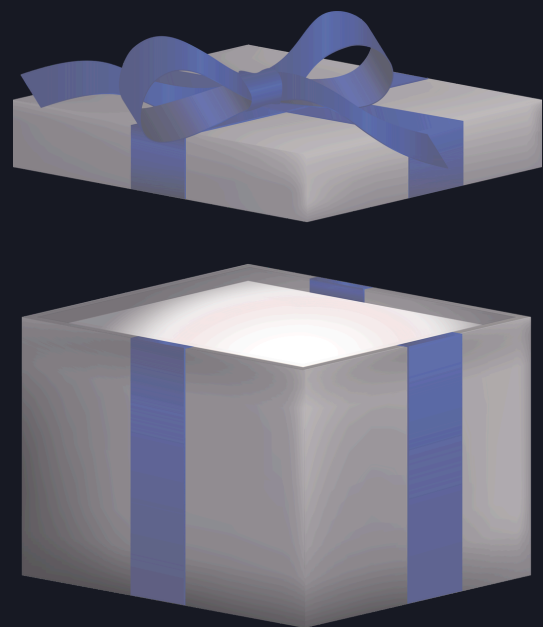
DECLARE (DECLARA EL CURSOR)

```
DECLARE nomb_cursor CURSOR FOR [SELECT...];
```



OPEN (ABRE EL CURSOR)

```
OPEN nombre_cursor;
```



FETCH (RECUPERA/AVANZA FILA(S))

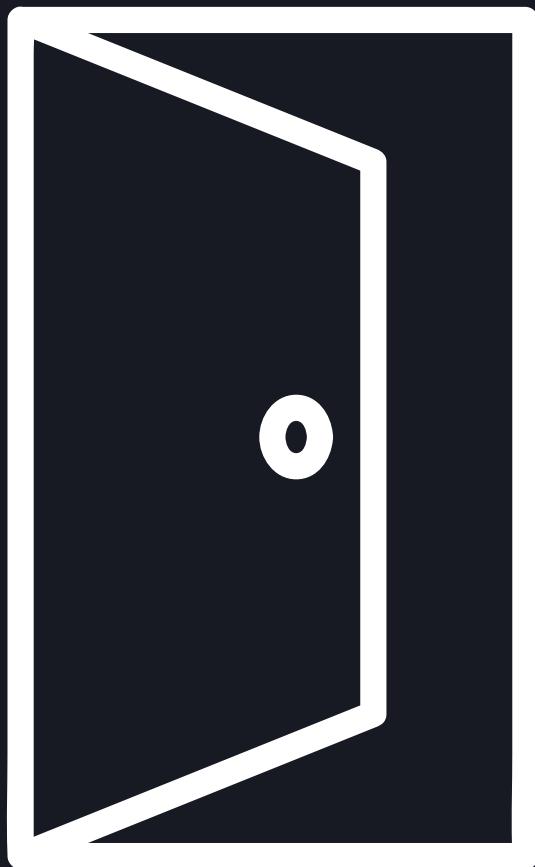
```
FETCH nombre_cursor INTO variables;
```


LOOP (CICLO DE ITERACIÓN)

LOOP

EXIT WHEN NOT FOUND (DETIENE EL CICLO)

EXIT WHEN NOT FOUND;



CLOSE (CIERRA EL CURSOR)

CLOSE nombre_cursor;

EJEMPLO DE USO

PRODUCTOS			
Producto_ID	Nombre	Decripción	Precio
1	Ensalada	Ensalada de frutas con lechuga	45
2	Pasta	Pasta tradicional	60.5
3	Hamburguesa	Clasica con papas	55.5
4	Papas	Orden de papas fritas	39.99
5	Pizza	De peperoni con extra queso	64.99
6	Burrito	Burrito de carne con papas	54.99

```

DECLARE
    total NUMERIC(6,2) := 0;
    aux NUMERIC(4,2);

    aumento CURSOR FOR
        SELECT precio
        FROM productos;

BEGIN

    OPEN aumento;

    LOOP
        FETCH aumento INTO aux;

        EXIT WHEN NOT FOUND;
        total := total + aux;
        RAISE NOTICE 'Precio: %    Acumulado: %', aux, total;
    END LOOP;

    CLOSE aumento;

    RAISE NOTICE 'Total: %', total;

END

```

```

DECLARE
    id INTEGER;
    antiguo NUMERIC(4,2);
    nuevo NUMERIC(4,2);

    aumento CURSOR FOR
        SELECT producto_id, precio
        FROM productos;

BEGIN

    OPEN aumento;

    RAISE NOTICE 'MODIFICANDO PRECIOS';

    LOOP
        FETCH aumento INTO id, antiguo;

        EXIT WHEN NOT FOUND;

        IF antiguo > 60 THEN
            nuevo := antiguo*1.1;
        ELSE
            nuevo := antiguo * 1.2;
        END IF;

        RAISE NOTICE 'Antiguo: %    Nuevo: %', antiguo, nuevo;

        UPDATE productos
        SET precio = nuevo
        WHERE producto_id = id;

    END LOOP;

    CLOSE aumento;

END

```

```
DECLARE
    registro RECORD;
    nuevo NUMERIC(4,2);

BEGIN

    FOR registro IN
        SELECT producto_id, precio
        FROM productos
    LOOP

        nuevo := registro.precio * 1.2;

        RAISE NOTICE 'Producto ID: %', registro.producto_id;
        RAISE NOTICE 'Antiguo: %      Nuevo: %',
            registro.precio, nuevo;

        UPDATE productos
        SET precio = nuevo
        WHERE producto_id = registro.producto_id;

    END LOOP;
```

EJERCICIO

Se tienen los siguientes datos para alumnos a los cuales se les quiere modificar la beca para su siguiente semestre. Se requiere:

Crear un bloque DO que:

1. Declare un cursor para recorrer:

- alumno_id
- nombre
- promedio
- beca

2. Recorra todos los alumnos usando LOOP.

3. Si el promedio es mayor o igual a 9.0:

- aumentar la beca un 20%

4. Si el promedio es menor a 7.0:

- disminuir la beca un 10%

5. Mostrar mensajes con RAISE NOTICE indicando:

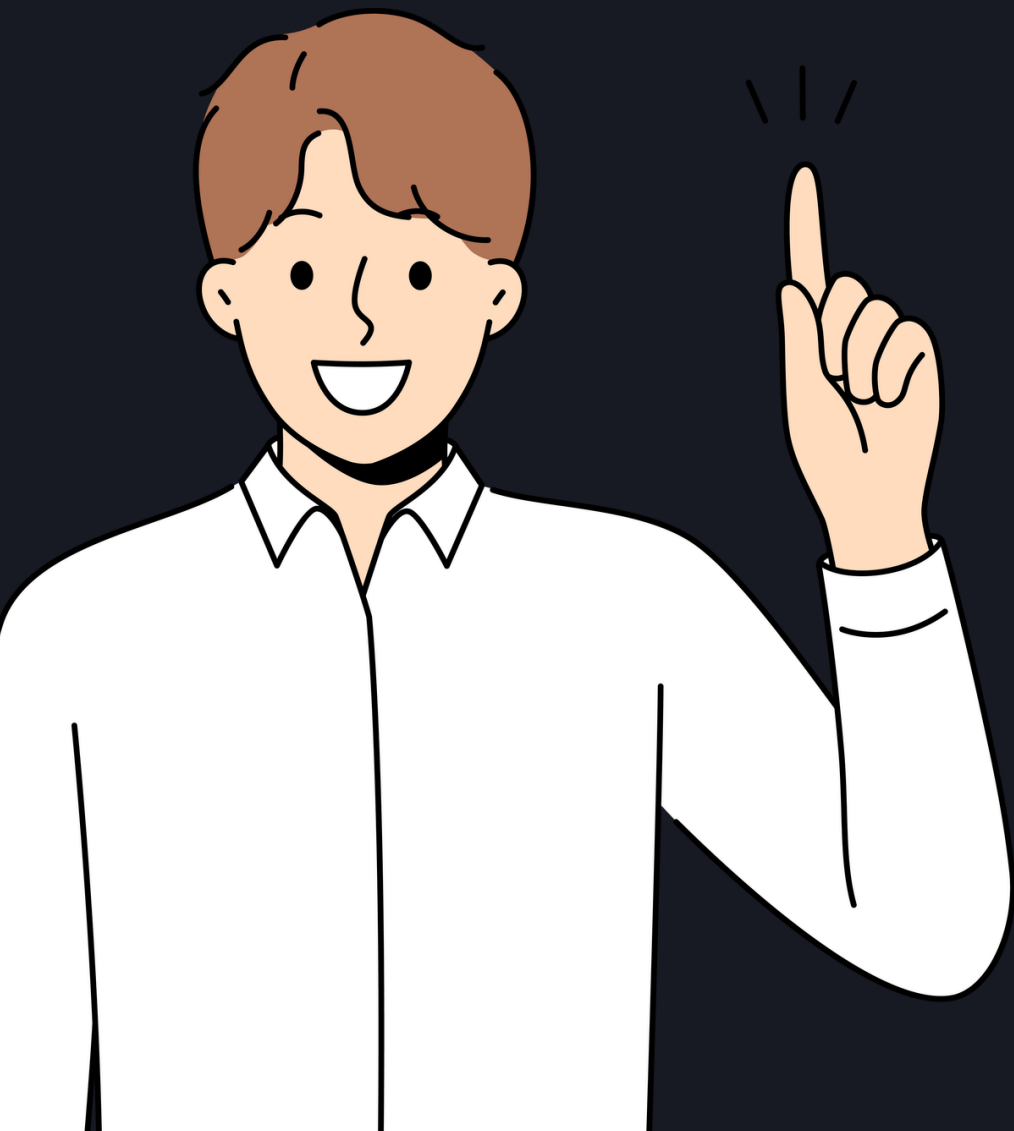
- nombre
- beca anterior
- beca nueva

6. Mostrar la tabla actualizada con un SELECT

```
CREATE TABLE alumnos (  
    alumno_id INTEGER PRIMARY KEY,  
    nombre VARCHAR(50),  
    promedio NUMERIC(3,1),  
    beca NUMERIC(6,2)  
);
```

```
INSERT INTO alumnos VALUES  
(1, 'Ana', 9.5, 2000),  
(2, 'Luis', 7.8, 1500),  
(3, 'Carlos', 8.9, 1800),  
(4, 'Maria', 9.8, 2500),  
(5, 'Elena', 6.9, 1000);
```

SOLUCIÓN PROPUESTA



```
DECLARE
  id INTEGER;
  nom VARCHAR(50);
  prom NUMERIC(3,1);
  beca_actual NUMERIC(6,2);
  nueva_beca NUMERIC(6,2);
  alumnos_cursor CURSOR FOR
    SELECT alumno_id, nombre, promedio, beca
    FROM alumnos;

BEGIN
  OPEN alumnos_cursor;
  LOOP
    FETCH alumnos_cursor
    INTO id, nom, prom, beca_actual;
    EXIT WHEN NOT FOUND;
    IF prom >= 9.0 THEN
      nueva_beca := beca_actual * 1.20;
      UPDATE alumnos
      SET beca = nueva_beca
      WHERE alumno_id = id;
      RAISE NOTICE 'Alumno: %', nom;
      RAISE NOTICE 'Beca anterior: % Nueva beca: %',
        beca_actual, nueva_beca;
    ELSIF prom < 7.0 THEN
      nueva_beca := beca_actual * 0.90;

      UPDATE alumnos
      SET beca = nueva_beca
      WHERE alumno_id = id;
      RAISE NOTICE 'Alumno: %', nom;
      RAISE NOTICE 'Beca anterior: % Nueva beca: %',
        beca_actual, nueva_beca;
    END IF;
  END LOOP;
  CLOSE alumnos_cursor;
END
```


Referencias

- [1] PostgreSQL Global Development Group, "Cursors," PostgreSQL Documentation, 2025. [Online]. Available: <https://www.postgresql.org/docs/current/plpgsql-cursors.html>.
- [2] GeeksforGeeks, "PostgreSQL Cursor," GeeksforGeeks, 2024. [Online]. Available: <https://www.geeksforgeeks.org/postgresql/postgresql-cursor/>. [Accessed: May 19, 2026].
- [3] Castillo R, "Cursor Es," Scribd. [Online]. Available: <https://es.scribd.com/document/950871261/Cursor-Es>